# Concrete Architecture Report of Gemini CLI

Mar 13, 2026

**Authors:**

Mehrin Hossain (23nt1@queensu.ca)
Lauren Sutherland (21las46@queensu.ca)
Jaineel Modi (22rsw2@queensu.ca)
Sophia Kim (18yjk@queensu.ca)
Minh Quang Tran (22rfx@queensu.ca)
Michael Okene (22gvl2@queensu.ca)

# Abstract

The Gemini CLI's concrete software architecture is detailed within this report. It compares this concrete architecture to that of the conceptual architecture provided in Deliverable A1. Through the use of the SciTools Understand application, the concrete dependency graph of the Gemini CLI's codebase was derived, analyzed, and used to establish the actual system boundaries. The results confirm that the system employs layered, client-server, and agent-oriented architectural patterns; however, the reflexion analysis demonstrates substantial architectural differences relative to the conceptual model. Of particular interest, the Task Delegation System serves as an Application-level concern in conjunction with the Orchestration Engine, rather than functioning as a lower level service. Additionally, the architecture deviates from strictly layered in order to provide human-in-the-loop safety; the Presentation layer (Terminal Interface), for example, holds a dependency directly upon the Infrastructure layer (Security & Execution Environment) to manage approval gates. Ultimately, the concrete implementation prioritizes both responsive interaction and modular safety, versus strict layered isolation.


# 1 Introduction

The purpose of this report is to describe the Concrete Architecture for the Gemini CLI and evaluate the degree to which it deviates from the Conceptual Design of the Gemini CLI. Although the Conceptual Architecture provides a high-level model for what the System intended to be, evaluating the Concrete Codebase will provide insight into the practical realities, trade-offs and runtime dependencies that are necessary to perform an AI-driven Command-Line Assistant. This Report describes the Derivation Process utilized to develop the architecture, presents both High Level and SubSystem Level Models, and evaluates Architectural Drift such as the relocation of Task Delegation to the Application Layer and the introduction of Cross-Layer Security Dependencies; finally, the Report describes the Concrete External Interfaces, Concurrency Models, and Step by Step Use Cases, to illustrate the system performing in operation.

# 2 Derivation Process

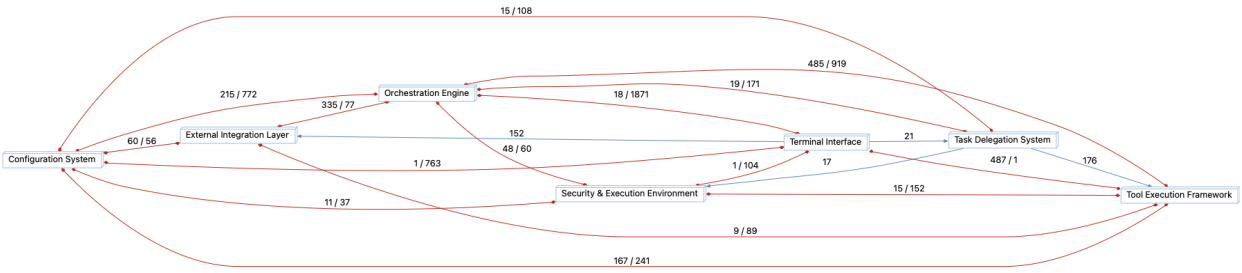


*Figure 1: Dependency Graph of the Gemini CLI Architecture*

The concrete architecture of the Gemini CLI was recovered using the SciTools Understand application. The concrete architecture is represented in the following figure. The concrete architecture was recovered using the SciTools Understand application. A pre-built .und project file containing the system's codebase was loaded into the Understand SciTool to recover a raw dependency graph. Using the Architecture

Designer, the codebase was systematically analyzed to group directories into major subsystems. As part of this mapping process, some conceptual assumptions were corrected to accurately represent the physical codebase. For example, the a2a-server was mapped to the Orchestration Engine subsystem, and the Task Delegation System was mapped to the Application Layer to accurately represent its tight coupling to core reasoning. To validate the accuracy of these groupings, the concrete dependencies were cross-referenced against public documentation, the GitHub repository, and core use-case sequence diagrams. Consistency passes were executed to remove granular utility classes and accurately represent the concrete architectural boundaries.

# 3 Concrete Architecture

## 3.1 Architecture Style

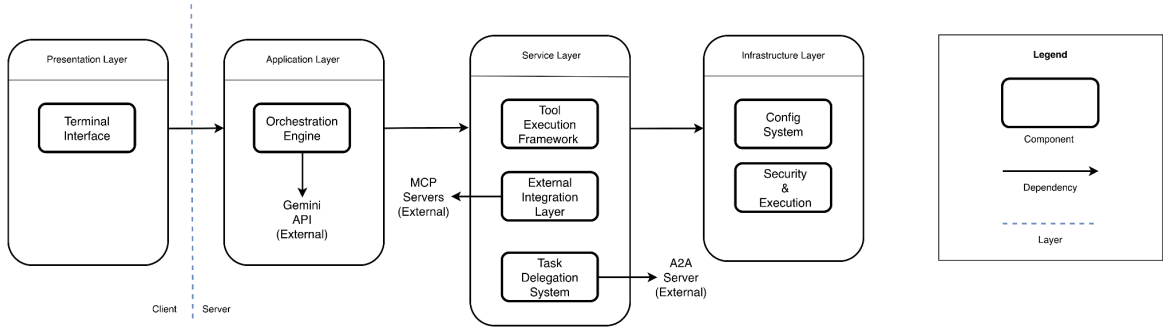


*Figure 2: Conceptual Diagram (based on global A1 feedback)*

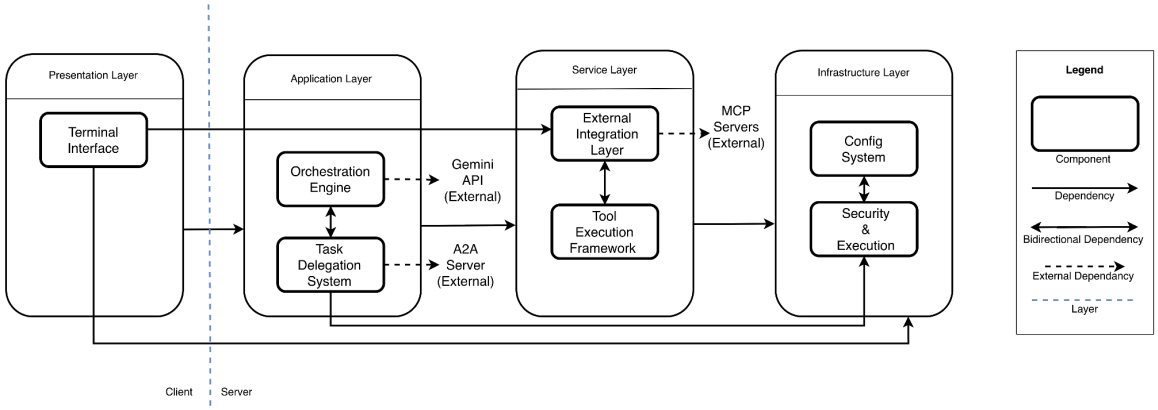


*Figure 3: Concrete Diagram*

The architecture of Gemini CLI primarily follows a layered architecture style, where separate responsibilities promote modularity and a clear separation of concerns. The system also incorporates elements of client-server architecture and agent-oriented architecture. Firstly, client-server style architecture is represented as the local command-line interface communicates with the remote Gemini API to generate responses and perform reasoning tasks. Characteristics of Agent-oriented architecture in this system is present in the orchestration workflow, where the system interprets model outputs and coordinates actions such as tool invocation or task delegation, which enables the CLI to support multi-step workflows.

**The Presentation Layer** contains a single component, the Terminal Interface, which serves as the primary interaction point between the user and the Gemini CLI system. Through the command-line interface, users submit prompts, commands, or contextual files and receive responses directly in the terminal. This layer is responsible only for handling user interaction and formatting input and output. User requests are then forwarded to the Application Layer, ensuring that interface concerns remain separate from system logic and execution.

**The Application Layer** acts as the control hub of the system and includes the Orchestration Engine and the Task Delegation System. The orchestration engine manages the main execution workflow by receiving user input from the terminal, communicating with the external Gemini API, and interpreting model responses to determine the next step in the reasoning process. Depending on the output, it may generate a response, execute tools, or delegate tasks. The task delegation system supports this process by enabling the orchestration engine to distribute complex tasks to external agents or systems through mechanisms such as Agent-to-Agent (A2A) communication.

**The Service Layer** provides the operational capabilities that allow Gemini CLI to perform actions beyond prompt processing. This layer includes the Tool Execution Framework and the External Integration Layer. The tool execution framework enables the system to interact with the local environment by executing predefined tools such as file operations, shell commands, or content retrieval. The external integration layer extends these capabilities by enabling communication with external services such as Model Context Protocol (MCP) servers, allowing the system to access additional tools and resources without modifying the core logic.

**The Infrastructure Layer** provides the foundational components required for safe and configurable system execution. This layer includes the Configuration System and the Security and Execution Environment. The configuration system manages runtime behavior through configuration files, environment variables, and project context documents. The security and execution environment enforces permission checks, authentication, and execution constraints to ensure that system actions and tool operations occur within controlled boundaries. Together, these components ensure reliable and secure operation across the system.

## 3.2 High-Level Architecture Subsystems

The concrete architecture of Gemini CLI consists of 6 major components and its various subcomponents. Spread across four different layers, the grouping of each component and its subcomponents denotes a clear distinction in logic and functionality, which can be further explored to see its cross-component and high-level interactions.

### 3.2.1 Terminal Interface

The terminal interface is the user interaction manager that handles all aspects of user experience. The main subcomponents that make this possible are:

- **Input Manager** - This subcomponent is used to parse user input, break down the commands, and handle these commands accordingly. It mainly operates as the UI for much of the backend logic, including that for external packages, hooks, remote tools/data servers, and skills [3].

- **Renderer** - The renderer manages visual rendering/layout, which includes styling of code blocks, headers, progress animations, colours/styles, and other user-defined UI settings.
- **Response Formatter** - This subcomponent is important for ensuring that the output is human readable and has been properly translated into an acceptable structure.
- **UI Core** - The UI core handles the initialization and setup of the environment, as well as cleanup. This subcomponent also manages the state of the system to keep track of its execution for the input manager and renderer. It serves as the bridge between the UI services and the backend [1].
- **UI Services** - This is a subcomponent that supplies background supports and utilities to the UI. It is used by the input manager to generate information needed to execute its commands. For example, the UI service component may be responsible for discovering and loading all available prompts from all configured MCP servers and adapting them into executable SlashCommand objects for the input manager to use [10].

### 3.2.2 Orchestration Engine

The orchestration engine is responsible for the decision-making, prompt construction, coordination, and context creation and management. It is the focal point of the backend as an intelligence coordinator. It consists of the following subcomponents:

- **Core Orchestrator** - This subcomponent manages short-term conversation context/history and the state of the user's request in the system's lifecycle [3]. It provides "hooks" where prompts or tools can be intercepted or blocked during execution [5]. The subcomponent also orchestrates the instantiation of the agent-to-agent gateway through which the task delegation system can enable intercommunication between AI agents.
- **Core Utility** - Handles monitoring and metrics pertaining to the availability of the Gemini API and other remote servers. It informs the model manager about the state of the AI model (ie. token count). Core utility also manages the low-level tools/logic used by the backend, while keeping it centralized in the application layer for bundled access [3].
- **Model Manager** - This subcomponent manages the fallback and routing of the system to the AI models. It uses predefined strategies that allow the system to analyze the user's request and route it to the Flash model (simple) or the Pro model (complex) based on complexity. Model fallback enables the component to assess the information provided by core utility regarding token count and model availability, and switch the user from the default pro model to the flash model to maintain system availability [10].
- **Prompt Constructor** - Uses system configurations and rules, as well as the user's request and session/system history to craft a prompt for the Gemini API. By using context injection, it is able to create safe and optimized prompts for the model [10].

### 3.2.3 Task Delegation System

The task delegation system handles the agent and sub-agent entities that are instantiated and operated by the system to handle specialized tasks and workflows. This allows for autonomous execution procedures that can extend agent capabilities.

- **Agent Manager** - This subcomponent manages and executes from a centralized registry of subagents, as well as providing the option to make custom subagents. The main Gemini agent can

initialize these sub-agents to handle specific, specialized tasks by allowing the orchestration engine to call an agent as if it were a tool [2]. Each subagent possesses its own system prompt and persona, access to specialized tools, and allows subagent interactions to happen within an independent context window [3].

- **Agent Workflows** - The subcomponent provides the instructions, resources, and scripts that form context and procedural knowledge into executable directives for agents. Skills provide agents with specialized knowledge that can be packaged into reusable multi-step instructions for repeatable workflows with domain knowledge for new capabilities [2].

#### 3.2.4 External Integration Layer

This component manages interactions with external systems, tools, and data sources, which includes real-time connectivity between Gemini CLI and the user's code editor or with third-party servers and services, to extend Gemini CLI's capabilities beyond its built-in features.

- **IDE Integration** - This subcomponent is responsible for the seamless integration between the user's IDE and Gemini CLI in order to integrate Gemini models directly into the developer's codebase to help understand code, build workflows and automate tasks within that project's context. It gathers context about the IDE, it creates a bridge between the CLI and the IDE, and integrates Gemini CLI capabilities directly into the IDE such as being able to accept changes by the model which alters physical files in the codebase [6].
- **MCP Implementation** - Implements the MCP Context Protocol to connect other external systems like other AI applications, data and file systems, development tools and productivity tools [1]. This subcomponent manages communication to remote MCP servers, including the authentication and tool discovery/formatting for the Gemini CLI [7].

#### 3.2.5 Tool Execution Framework

The Tool Execution Framework executes tools commands, manages task synchronization, and other services that provide functionality for tools. Its main subcomponents are:

- **Data Sources** - Manages a static directory of data sources that can be used by both the Gemini CLI and external services as context [10].
- **Tool Execution** - Handles the lifecycle of a tool's execution and the high-level logic for this framework. The built-in scheduler manages parallel tool calls, checks policy surrounding tools, and keeps track of the tool's state [9].
- **Tool Services** - Provides the low-level capabilities for this component by providing services/logic that different tools need. The tools registry calls services to provide extensible, complex functionality for tools across several different domains.
- **Tool Registry** - This subcomponent consists mainly of tool definitions. Aside from containing local tools, the tool registry also helps discover new tools (primarily from extensions or external services) that the system did not originally know about at compile-time [9].

#### 3.2.6 Configuration System

This component manages global memory, session history and management such as settings validation. It provides the low-level structure to the whole system.

- **Memory Manager** - Reads and writes to a centralized memory core that stores user preferences and conversation history. It helps provide the system with context for future interactions into a comprehensive GEMINI.md file. Handles the initialization of a new memory core upon installation and can also restore previous conversation history [10].
- **Session Maintenance** - Focuses more on the rules and settings that determine the system's configuration. As the information core of the project, it processes possible conflicting settings/instructions from the default values, environment variables, project settings, and user settings [3]. It contains specific rules for various scenarios such as which model to fallback on and initializes default instances and values [10]. This subcomponent also manages a dynamic list of AI models and their relevant information.

#### 3.2.7 Security & Execution Environment

The Security & Execution Environment component ensures safety, consistency and user approval for actions. This component is responsible for the enforcement of rules from the Config. System.

- **Approval Gate** - This subcomponent interrupts the execution of operations that require user approval such as shell executions or file modifications [1]. It freezes the execution loop until the user has provided an answer, which allows for controlled handling by the user [10].
- **System Enforcement** - Uses the policy engine to check a tool call against a set of rules that determine whether a tool is allowed, denied, or requires permission (which would then require the use of the Approval Gate) [3]. It also implements the standard built-in safety rules from Google/Gemini API. This subcomponent also builds context objects that include prompts, responses, tool outputs, requests, etc. against which the safety checkers scan for either pre-determined or custom checks [9]. System Enforcement manages a list of these safety checkers and coordinates its decisions and resulting actions.

## 3.3 Reflexion Analysis

#### 3.3.1 Reflexion Analysis Between Conceptual (Figure 2) and Concrete Design (Figure 3)

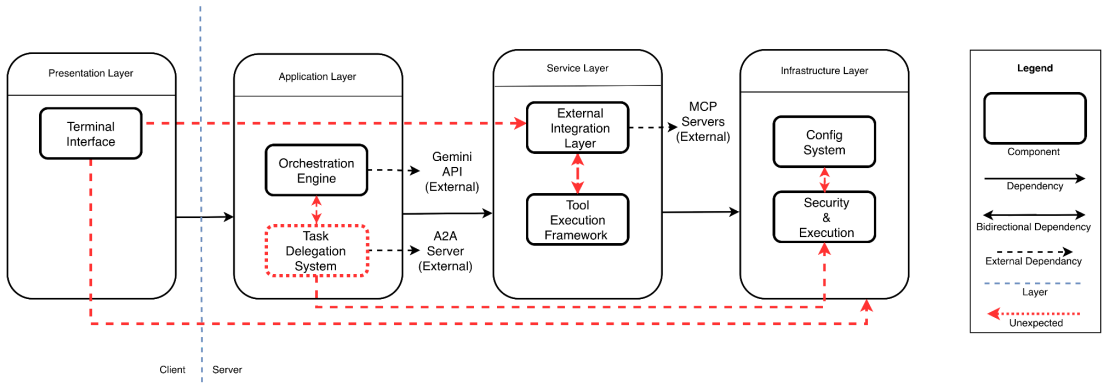


*Figure 4: Reflexion of Concrete Architecture Diagram (changes from conceptual diagram shown in red)*

The concrete diagram shows a dependency from the **Terminal Interface to the Infrastructure Layer**, specifically the Security & Execution Environment. In the conceptual architecture, the

presentation layer was expected to interact only with the application layer. However, the implementation indicates that the CLI interface must interact with infrastructure components when actions require user approval. For example, operations such as file modification or shell execution may trigger approval prompts that are displayed directly in the terminal. Because the terminal interface handles these prompts and collects user confirmation, it must communicate with the execution environment during runtime [4].

Another divergence appears between the **Terminal Interface and the External Integration Layer.** Conceptually, external services were expected to be accessed only through the application layer. In practice, the terminal interface interacts with integrations to discover available prompts and commands provided by Model Context Protocol (MCP) servers. Gemini CLI dynamically loads prompts and exposes them as slash commands in the terminal, which requires the interface to query external integrations during initialization or command discovery [10].

The **Task Delegation System is moved to the Application Layer** in the concrete architecture rather being in the service layer as originally modeled. This change reflects the close relationship between delegation and orchestration logic. The orchestration engine determines when tasks should be delegated to specialized agents or workflows and coordinates their execution within the reasoning loop [2].

The reflexion model also shows a **bidirectional dependency between the Orchestration Engine and the Task Delegation System**. The orchestration engine initiates delegated tasks, while delegated agents return results that influence the system's reasoning and workflow state. Because these components exchange control and context during execution, a mutual dependency emerges between orchestration and delegation modules [10].

The reflexion diagram depicts a dependency from the **Task Delegation System to the Security & Execution Environment** in the infrastructure layer. Conceptually, infrastructure components were expected to support services passively without direct dependencies from application-level components. However, delegated tasks frequently trigger operations such as tool execution, file access, or shell commands, which must pass through the system's security enforcement mechanisms. Because delegated workflows may invoke tools that interact with the local environment, the delegation system must communicate with the execution environment to ensure these actions comply with configured safety policies and permission checks [10].

Within the Service Layer, the concrete structure shows a **bidirectional dependency between the External Integration Layer and the Tool Execution Framework.** External integrations expose tools discovered from MCP servers or extensions, while the execution framework is responsible for registering and running these tools within the CLI environment [10]. Since integrations supply tools and the execution framework executes them, the two components depend on each other during runtime.

Finally, the **Configuration System and the Security & Execution Environment show a bidirectional dependency** in the infrastructure layer. Configuration settings define execution policies, tool permissions, and environment behavior, while the security system enforces these policies during tool execution [10]. Because policy enforcement requires configuration data and configuration must interact with the execution environment to apply these rules, a mutual dependency exists between the two components [8].

No major **absences** were observed, indicating that the conceptual architecture successfully captured most structural relationships within the system. Overall, the reflexion analysis demonstrates that Gemini CLI largely follows the intended layered architecture. The observed divergences mainly arise from practical runtime requirements of an interactive AI agent system, particularly the coordination between orchestration logic, tool execution, external integrations, and runtime configuration.

### 3.3.2 Reflexion Analysis Between Original (Figure 5) and New (Figure 2) Conceptual Design

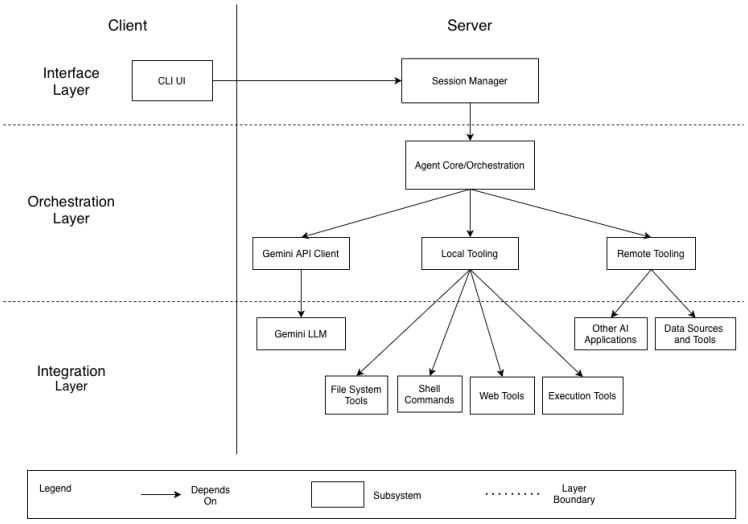


*Figure 5: Original Conceptual Architecture Diagram From Assignment 1*
*(See Figure 2 to see our new Conceptual Diagram for Assignment 2)*

After reading the global feedback, we made some changes to our conceptual architectural diagram. Both diagrams still use a layered architecture style with elements of client-server but the new version omits the lower-level integration layer and adds a service and infrastructure layer. This change is based on the global feedback to focus on the architectural layers of GeminiCLI rather than the lower level components, as well as to reflect the systems responsible for configuration and security that were overlooked in our original design. The interface layer with CLI UI and a session manager got simplified to a presentation layer with a terminal interface to represent the client side of the system. Another deviation sees the orchestration layer being turned into the application layer, which still consists of a core orchestration engine, but some of the other components in the original orchestration layer are moved into the new service layer. Separating these functionalities into different layers emphasizes the modular nature of the system. This segmentation is further explored and improved in our concrete design, as seen above. The infrastructure layer was added to the new diagram to distinguish the services and functionality responsible for security and system configuration, which had either previously been missed in our original design or had been grouped under Agent Core.

# 4 Tool Execution Framework and Interactions

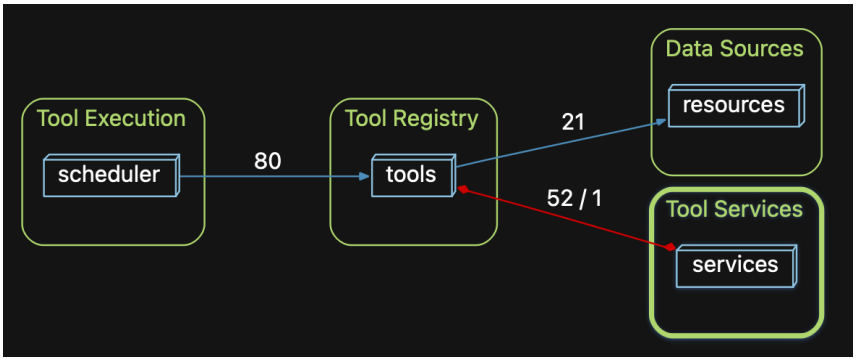


*Figure 6: Dependency Graph of Tool Execution Framework and its Subcomponents*

This section describes the concrete architecture of the Tool Execution Framework (TEF), a subsystem within the Service Layer responsible for executing tools and interacting with system resources. Unlike components that focus on reasoning or workflow coordination, the TEF manages the operational aspects of tool invocation, including tool definitions, execution scheduling, and access to required resources [4][9].The TEF consists of four primary components: Tool Execution, Tool Registry, Tool Services, and Data Sources. The Tool Registry maintains available tool definitions, the Tool Execution subsystem manages runtime tool execution, the Tool Services component provides operational logic used during execution, and the Data Sources component provides the resources used by tools during operation [9].In the repository structure, these components correspond to the tools, scheduler, services, and resources modules, where tool definitions rely on execution services and resource interfaces, while the scheduler coordinates their runtime execution [10].

## 4.1 Tool Execution

The Tool Execution subsystem manages the lifecycle and coordination of tool operations. This subsystem ensures that tools are executed in an organized and controlled manner while maintaining system responsiveness [9]. In the concrete implementation, this functionality is primarily implemented through the scheduler module [10].

The scheduler module coordinates the execution of tools and manages their state. Key files within this module include scheduler.ts, tool-executor.ts, tool-modifier.ts, and state-manager.ts. The scheduler is responsible for determining when a tool should be executed, maintaining the execution state of active tools, and enforcing policies related to tool invocation [10]. For example, the tool-executor.ts component handles the runtime invocation of tools, while state-manager.ts maintains information about tool execution states and task progress . Policy enforcement within the scheduler ensures that tools follow system constraints and execution rules [8][9].

The scheduler interacts with these services during tool execution. When a tool is invoked, the scheduler resolves the appropriate execution path and delegates the operational work to the relevant service implementation [9][10].

## 4.2 Tool Registry

The Tool Registry maintains the definitions of all tools available to the Gemini CLI. This component acts as the central catalogue of executable capabilities and enables the system to dynamically discover and reference tools during runtime [9]. Within the repository structure, tool directory contains numerous tool implementations, including files such as glob.ts, grep.ts, read-files.ts, shell.ts, and mcp-tool.ts [10]. Each file defines a specific tool and describes how it interacts with the system.

The glob tool (glob.ts) provides pattern based file discovery within the project directory. It allows the system to locate files that match a specified pattern, such as searching for files with a specific extension or within a specific directory structure. This capability is usually useful for code analysis workflows, where the system may need to identify all relevant files before performing further processing [9][10].

The grep tool (grep.ts) enables text searching within files. While its similar to the traditional Unix grep command, this tool allows the system to scan files for occurrences of specific text patterns. This functionality is useful for tasks such as locating functions, variables, or configuration entries across a codebase. By incorporating a grep style search capability into the tool registry, the system can efficiently analyze large code repositories [9][10].

The read-files tool (read-files.ts) provides functionality for retrieving the contents of files. This tool allows the system to load file data into memory so that it can be analyzed or processed further. In many workflows, file discovery tools such as glob are used first to identify relevant files, after which the read-files tool retrieves their contents for inspection or transformation [9][10].

The shell tool (shell.ts) allows the system to execute shell commands within the controlled execution environment. This capability enables Gemini CLI to perform operations such as running scripts, invoking system utilities, or interacting with development tools installed on the local machine[9]. Because shell execution can potentially modify the system environment, it is typically used in conjunction with security and execution policies defined elsewhere in the architecture[8] [9].

The MCP tool (mcp-tool.ts) supports integration with external tools provided through the Model Context Protocol. This component allows Gemini CLI to dynamically discover and invoke tools exposed by external MCP servers [7][11]. By integrating MCP tools into the registry, the system can extend its capabilities beyond the built-in tools defined within the local repository [7].

These tool modules encapsulate both the metadata and execution logic required for a given operation. The Tool Registry enables the framework to dynamically resolve tool identifiers during execution. When a tool is requested, the scheduler references the registry to determine which tool implementation should be executed and which services are required to support the operation [9][10].

## 4.3 Data Sources

The Data Sources subsystem provides the resources that tools interact with during execution. Instead of allowing tools to directly access system resources, the architecture centralizes resource management within a dedicated component [4]. This design improves maintainability and provides a consistent

interface for accessing external data.In the concrete implementation, this subsystem corresponds to the resources module [10]. A key component of this module is the resource-registry.ts file, which maintains a collection of discoverable resources available to the system [10]. These resources may include local files, project data, or external resources provided through integrations [7]. The resource registry enables both local tools and external integrations to reference available data sources in a consistent way [4][7]. For example, tools defined within the Tool Registry may reference resources stored in the registry in order to access project files or contextual data during execution [9][10]. By isolating resource access within this subsystem, the architecture ensures that tools interact with resources through standardized interfaces rather than directly accessing system level APIs [4]. This separation also enables additional safety checks or validation mechanisms to be applied when resources are accessed [8].

## 4.4 Tool Services

The Tool Services subsystem provides the low level operational capabilities required by tools during execution. Instead of embedding system operations directly within the scheduler or tool implementations, these responsibilities are delegated to specialized service modules, improving modularity and separation of concerns [4]. In the concrete implementation, this subsystem corresponds to the services directory, which includes modules such as fileSystemService.ts, fileDiscoveryService.ts, gitService.ts, contextManager.ts, shellExecutionService.ts, environmentSanitization.ts, loopDetectionService.ts, modelConfigService.ts, and sessionSummaryService.ts [10]. These services provide functionality for interacting with the filesystem, repositories, execution environments, and contextual data used by tools. During runtime, the Tool Execution subsystem invokes Tool Services to perform the operational work required by a tool. By isolating these capabilities within a dedicated subsystem, the architecture allows new functionality to be added through service modules without modifying the execution controller [4][9].

## 4.4 Integration and Modularity of the Tool Execution Framework

The Tool Execution Framework (TEF) is designed with a modular structure that separates tool definitions, execution logic, and resource management into independent subsystems. Its four main components each handle a specific stage of the tool lifecycle [4]. Integration occurs through a clear execution pipeline. The Tool Registry stores tool definitions and metadata, including both built-in tools and those discovered through external integrations such as MCP servers [7][9]. When a tool is invoked, the Tool Execution subsystem resolves the tool through the registry and manages its execution using the scheduler and service modules [9][10]. These services then interact with the Data Sources subsystem to access the files, system resources, or contextual data required by the tool [10].This separation keeps tool implementations loosely coupled from execution control and resource access. As a result, new tools can be added to the registry without modifying the scheduler, and new resources can be introduced without changing existing tool implementations [4]. The framework can also integrate external tools through the same execution infrastructure used by built in tools [7][9].Overall, this modular architecture improves maintainability, extensibility, and separation of concerns by organizing tool definitions, execution control, and resource access into clearly defined components [4].

## 4.5 Reflexion Analysis

The conceptual design presents Local Tooling and External Tooling as distinct subsystems responsible for local (filesystem/shell) and MCP-based (data sources/other AIs) interactions routed by Agent Core/Orchestration but the concrete implementation reveals a TEF where both local built-in tools and MCP-bridged tools form through a single plugin registry (tool-registry.ts) and execution pipeline. Additionally, the TEF considers persistent memory as a tool (memoryTool.ts) which was absent from the earlier design. The conceptual design described safety as centralized "sandboxing precautions" managed by the Tooling subsystem. However, in the concrete TEF safety responsibilities are distributed across individual tools: the Shell Execution Tool routes through an Approval Gate while Filesystem tools manages dependencies for safe file patching and user confirmation. This approach allows precise, auditable security controls that scale with each tool's specific risk profile. Lastly, extensibility was pitched through MCP adoption benefits but concretely realized through the MCP Tool Bridge that dynamically wraps remote capabilities as native TEF tools without modifying registry or orchestration logic.

# 5 Use Cases

**Legend:**

- **Solid message arrow:** Request/command being issued (forward request, request tool call, dispatch read or run command, display diff)
- **Dashed message arrow:** returned data/result (file contents, tool results, exit code, final summary**)**
- **Vertical dashed lines (lifelines):** each participant’s timeline; time flows top to bottom

## 5.1 Use Case 1: Refactor a File

**Use Case Summary:** A developer asks Gemini CLI to refactor a specific file. The Terminal Interface loads context from the Config System and submits the request to the Orchestration Engine. The Orchestration Engine calls the Gemini API (external) with the prompt and available tools. The Gemini API requests a file read, the Orchestration Engine checks authorization through Security & Execution, and the Tool Execution Framework executes read_file(fileY) and returns the file contents. Using those contents, the Orchestration Engine calls the Gemini API again to produce a proposed diff/patch, which the Terminal Interface displays to the user.

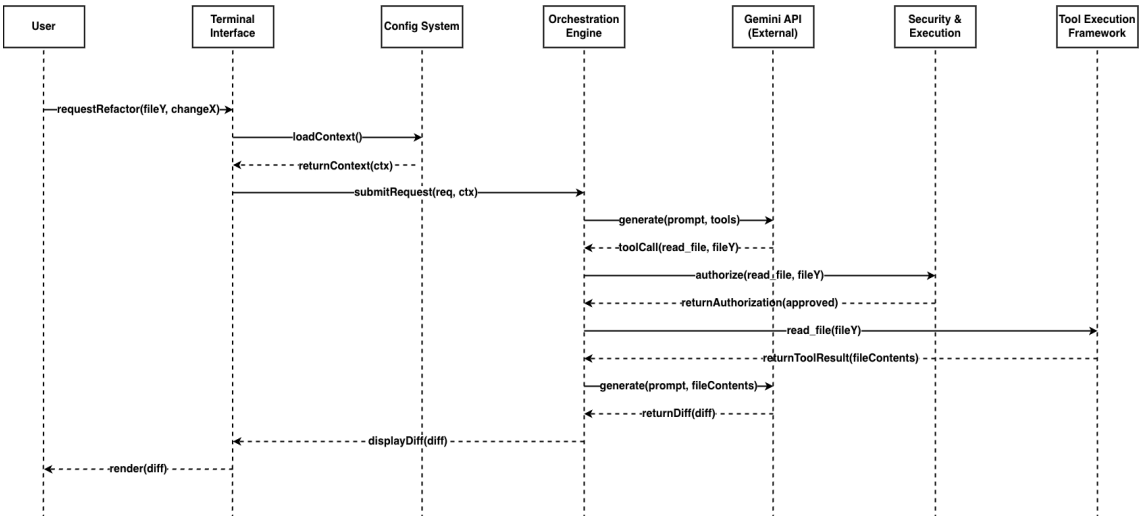

***Figure 7:*** *Sequence Diagram for Use Case #1 (Refactor X in File Y)*

**Explanation:** This use case refines the more abstract workflow shown in A1 by making the concrete subsystem interactions explicit. The Terminal Interface receives the refactor request, loads context from the Config System, and submits the request to the Orchestration Engine. The Orchestration Engine calls the Gemini API with the prompt and tool list, receives a tool call to read the file, and checks authorization through Security & Execution. If approved, the Tool Execution Framework runs read_file(fileY) and returns the file contents. The Orchestration Engine then calls the Gemini API again to generate the diff, and the Terminal Interface displays it. Compared to the conceptual version, this concrete use case makes the approval step, tool execution path, and function-level interactions visible. Lifelines match the A2 concrete architecture, and message labels use concrete methods recovered via Understand.

## 5.2 Use Case 2: Run Tests Safely and Summarize Failures (Approval + Sandboxed Shell)

**Use Case Summary:** A developer asks Gemini CLI to run the test suite. The Terminal Interface loads context from the Config System and submits the request to the Orchestration Engine. The Orchestration Engine calls the Gemini API (external), which proposes a test command. The Terminal Interface presents the command for approval. The Orchestration Engine checks the decision through Security & Execution. If approved, the Tool Execution Framework runs runCommand(cmd) via shell execution and returns stdout/stderr and an exit code, which the Orchestration Engine sends back to the Gemini API to generate a short diagnosis and next steps. If denied, the workflow stops and the Terminal Interface returns a cancellation response.

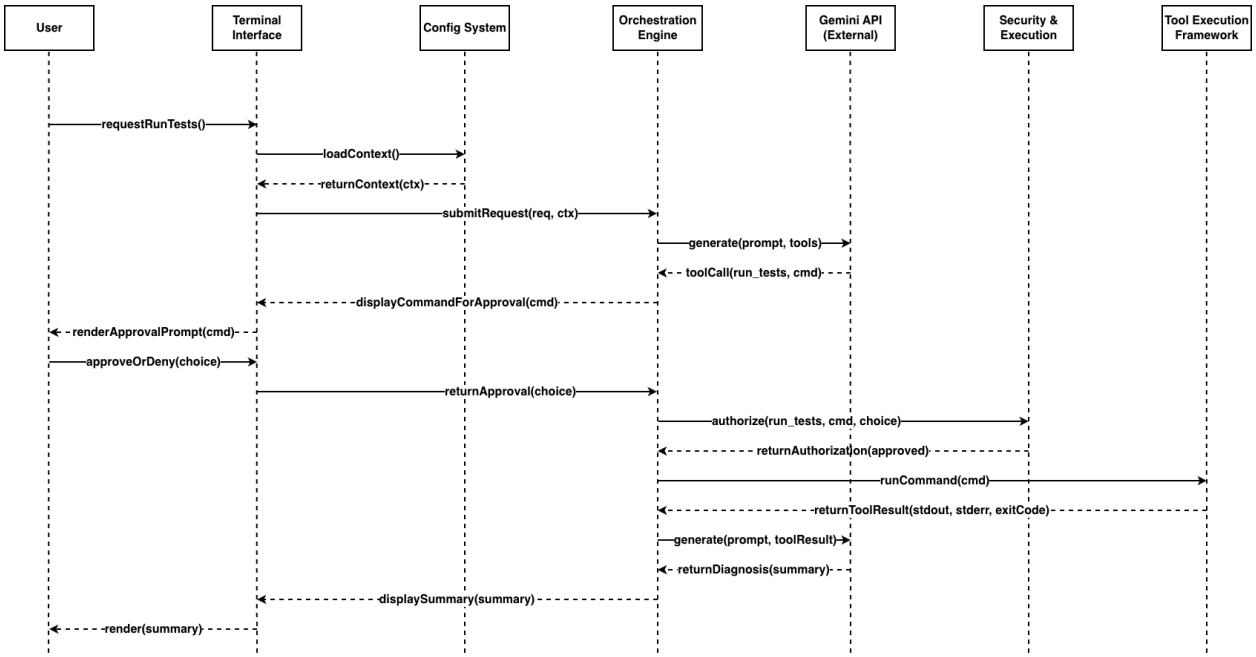


***Figure 8:*** *Sequence Diagram for Use Case #2 (Run Tests & Summarize Failures)*

**Explanation:** This use case also makes the A1 conceptual workflow more concrete by showing the exact approval and execution path. The Gemini API can suggest a test command, but execution only happens through the Tool Execution Framework after approval. The Terminal Interface loads context from the Config System, submits the request to the Orchestration Engine, and shows the proposed command for approval. The Orchestration Engine checks the decision through Security & Execution, then either stops the workflow if the command is denied or dispatches runCommand(cmd) if it is approved. The Tool

Execution Framework returns stdout, stderr, and an exit code, which the Orchestration Engine sends back to the Gemini API to produce a short diagnosis. Compared to the conceptual use case, this version makes the human-in-the-loop approval gate, shell execution path, and concrete method calls explicit. Lifelines match the A2 concrete architecture, and message labels use concrete methods recovered via Understand.

# 6 Conclusion

The concrete architecture of Gemini CLI still follows the main structure laid out in A1, but the real implementation is messier than the original conceptual view suggested. After analyzing the dependency graph in Understand, checking the repository, and comparing the results against the use cases, it became clear that the layered, client-server, and agent-oriented patterns are still present, just with tighter connections between components than expected. The biggest differences came from interactive terminal streaming, tool execution, task delegation, and runtime policy enforcement, all of which pull parts of the system closer together in practice. Even with those divergences, the architecture still preserves an important boundary between orchestration and execution. That separation helps Gemini CLI stay modular, keeps tool use more controlled, and supports the human-in-the-loop safeguards central to how the system works.

# 7 Lessons Learned

Moving from the conceptual architecture overview into a concrete design forced the Gemini CLI architecture to adhere to specific implementation details that taught the team a few things about architecture as a whole. The first realization was that strict layered architecture designs fall apart quickly under real-world application safety requirements. To enforce human-in-the-loop safety constraints, the Terminal Interface had to depend directly on the Security & Execution Environment's approval gate, skipping both the Application layer and Service layer. Second, the team realized that treating task delegation as a fundamental reasoning capability of agent-oriented systems rather than just a downstream service was necessary to justify elevating the Task Delegation System into the Application layer. Last, this concrete architecture reminded the team that modularity comes at a cost. While MCP integrations allow for ludicrous amounts of modularity, over-dependence on remote API calls causes heavy latency and near elimination of any offline capabilities.

# 8 References

[1] Alateras, J. (2025, July 8). *Unpacking the Gemini CLI: A high-level architectural overview*. Medium https://medium.com/@jalateras/unpacking-the-gemini-cli-a-high-level-architectural-overview-99212f6780e7

[2] Google. (2024). *Agent Skills*. Gemini CLI. https://geminicli.com/docs/cli/skills/

[3] Google. (2024). *Gemini CLI configuration*. Gemini CLI. https://geminicli.com/docs/reference/configuration/

[4] Google. (2024). *Gemini CLI documentation*. Gemini CLI. https://geminicli.com/docs/

[5] Google. (2024). *Gemini CLI hooks*. Gemini CLI. https://geminicli.com/docs/hooks/

[6] Google. (2024). *IDE integration*. Gemini CLI. https://geminicli.com/docs/ide-integration/

[7] Google. (2024). *MCP servers with the Gemini CLI*. Gemini CLI. https://geminicli.com/docs/tools/mcp-server/

[8] Google. (2024). *Policy engine*. Gemini CLI. https://geminicli.com/docs/reference/policy-engine/

[9] Google. (2024). *Tools reference*. Gemini CLI. https://geminicli.com/docs/reference/tools/

[10] Google Gemini CLI. (2024). *Gemini CLI source code repository*. Github. https://github.com/google-gemini/gemini-cli

[11] Model Context Protocol. (n.d.). *What is the Model Context Protocol (MCP)?* https://modelcontextprotocol.io/docs/getting-started/intro

# 9 AI Report

### 1) AI Member Profile and Selection Process

The AI model we used was **OpenAI ChatGPT** running GPT 5.2. We chose ChatGPT because our biggest risk on A2 was inconsistency across artifacts, especially between the updated conceptual architecture, the high-level concrete architecture, the second-level subsystem analysis, the reflexion analysis, and the sequence diagrams. It helped us catch naming drift, missing handoffs, and unclear responsibility boundaries between the Terminal Interface, Orchestration Engine, Tool Execution Framework, External Integration Layer, Task Delegation System, Security & Execution Environment, and Configuration System. We also considered Google Gemini and Anthropic Claude. Gemini was thematically relevant, but we prioritized quick iteration and strict constraint following during diagram and report refinement, while Claude worked well for longer prose but was less aligned with our need for structure and consistency checking. Since A2 is focused on concrete architecture rather than only conceptual structure, we needed a tool that could help us compare artifacts, tighten wording, and keep the deliverable internally consistent.

### 2) Tasks Assigned to the AI Teammate

We delegated tasks that are repetitive and structure-heavy, where humans often miss small inconsistencies after multiple edits.

- Sequence diagram flow checks for both use cases. Reason: AI is good at spotting missing handoffs, unclear message direction, and steps that assign responsibility to the wrong subsystem.
- Naming consistency scans across artifacts. Reason: AI quickly flags mismatched subsystem names between diagrams, report sections, and slides.
- Readability tightening for diagram labels and surrounding text. Reason: AI can propose shorter labels and remove redundant messages while keeping meaning.
- Reflexion analysis wording support. Reason: AI can help phrase divergences and rationales more clearly after the team has already identified them from Understand, the repository, and documentation.
- Rubric alignment checks. Reason: AI can help check whether sections actually address the A2 requirements, especially for derivation process, second-level subsystem analysis, and report presentation.

**Boundaries:** Humans made all final decisions. We did not rely on AI for internal facts about Gemini CLI beyond what we could justify with course sources, repository evidence, or references.

### 3) Interaction Protocol and Prompting Strategy

One member acted as the primary driver for prompts to keep terminology stable. The rest of the team reviewed outputs and requested specific follow-up checks.

**Workflow**: We set a canonical vocabulary first, then reviewed one artifact at a time. We started with subsystem names, then the high-level architecture, then the second-level subsystem, then Use Case 1, then

Use Case 2, and finally the matching report paragraphs and presentation slides. We required the AI to stay at the architecture level and avoid invented subsystems.
**Example prompt:** Review this A2 sequence diagram for correctness at the subsystem level. Only use these lifelines: Terminal Interface, Configuration System, Orchestration Engine, Gemini API, Security & Execution Environment, and Tool Execution Framework. Identify missing handoffs, wrong responsibility assignments, and redundant messages. Keep labels short and stay at the architecture level.
**Refinement**: Early prompts were too open and produced extra components, old terminology from A1, and implementation detail that was too specific. We improved results by forcing a fixed lifeline list, banning invented subsystems, and requiring the same terms used in the report.

### 4) Validation and Quality Control Procedures

We did not accept suggestions because they looked right. Each change had to pass these checks:

- Mapping check: Every diagram lifeline and subsystem label had to map to a component described in the report or be labeled as an external system.
- Responsibility check: Terminal Interface handles interaction, Orchestration Engine controls workflow, Tool Execution Framework is the execution path for tools, and Security & Execution Environment governs approval and safety for risky actions.
- Control flow check: Both use cases had to match the same high-level concrete architecture and avoid implied side effects.
- Repository and Understand check: Any statement about dependencies, structure, or subsystem responsibilities had to be checked against the dependency graph, repository structure, and documentation.
- Human review pass: At least one teammate who did not author the original diagram reviewed the diagram and nearby text for contradictions and naming drift.

### 5) Quantitative Contribution to Final Deliverable

Our good-faith estimate is that ChatGPT contributed about 20 to 25 percent of the overall effort for A2, based on the parts where its suggestions directly changed diagram wording, report phrasing, or artifact consistency compared to sections we wrote and verified entirely as a team. The highest contribution was naming consistency checks, sequence diagram flow checking, and wording refinement across report sections and slides. Moderate contribution was rubric alignment checks and tightening explanations for reflexion analysis and lessons learned. Lowest contribution was architectural analysis, subsystem identification, interpretation of the Understand dependency graph, citations, and final writing decisions, which were primarily human work.

### 6) Reflection on Human AI Team Dynamics

AI saved time by speeding up consistency checks and suggesting clearer wording quickly. It also created extra work because outputs required validation and occasional rewrites to avoid unsupported claims or incorrect terminology. The AI helped brainstorming by offering alternative ways to explain subsystem boundaries, divergences, and use case flow, especially when we were trying to keep the report and presentation aligned. The main challenge was over-specific suggestions or drift back into A1 terminology. We handled this by tightening prompts, keeping a fixed vocabulary, and rejecting details we could not justify. The key lesson for future deliverables is to treat AI like a fast reviewer, constrain it early, and rely on human verification for correctness and course alignment.